

Aprendizaje Automático

Clase 39: Clustering

K-means, K-modes y K-prototypes

Pedro Luiz Ramos

Martes 29 de septiembre de 2026

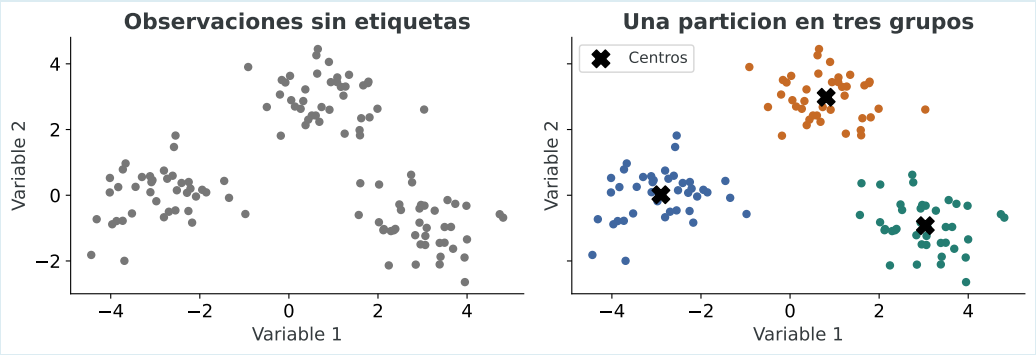
Clasificación y agrupación

Clasificación supervisada	Clustering
Observamos variables x y una clase conocida y.	Observamos variables x y buscamos una partición.
Aprendemos a predecir la clase de nuevos casos.	Describimos similitudes y diferencias entre grupos.
Logística, SVM, Naive Bayes.	K-means, K-modes, K-prototypes.

La etiqueta de un clúster se construye a partir de los datos.

El grupo 1 no significa “positivo”, “mejor” ni “mayor riesgo”.

El resultado de un agrupamiento



Obtenemos una etiqueta por observación y un perfil de cada grupo.

Ejemplo simulado. Las cruces representan los centros de K-means.

Similitud y propósito

Una misma base puede producir agrupamientos distintos según las variables.

- En clientes: frecuencia de compra, gasto o canal preferido.
- En vinos: mediciones químicas, sin una nota de calidad observada.
- En documentos: una representación de sus contenidos.

Primero definimos qué significa que dos observaciones sean parecidas.

Un grupo compacto puede ser poco útil para el propósito del análisis.

El método depende de las variables

Método	Variables	Representante
K-means	Numéricas	Media por variable
K-modes	Categóricas	Moda por variable
K-prototypes	Mixtas	Medias y modas

Los tres requieren fijar el número de grupos k.

El costo depende de la representación y de la disimilitud elegidas.

Datos: 177 vinos y 13 mediciones

El archivo del curso contiene `Codigo` y 13 variables numéricas.

- Alcohol, ácido málico, ceniza y alcalinidad.
- Magnesio, fenoles, flavonoides y otras mediciones.
- Color, matiz, OD280/OD315 y prolina.

`codigo` identifica cada fila y se excluye del ajuste.

Usamos las mismas 177 filas del material. No agregamos etiquetas de clase.

Adaptación docente de Wine, UCI, con 178 filas en la fuente original. En el archivo recibido hay 177.

https://doi.org/10.24432/C5PC7J. La recomendación comercial es un escenario hipotético.

Estandarización

Si una variable tiene una escala mucho mayor, puede dominar las distancias.

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

Ejemplo hipotético: alcohol de 14, media 13 y desviación 0.8.

$$z = \frac{14 - 13}{0.8} = 1.25.$$

Ese vino está 1.25 desviaciones sobre la media de alcohol.

Estandarizar iguala las escalas. No elimina atípicos ni correlaciones.

Datos y escala en R

```
library(cluster)
library(klaR)
library(clustMixType)
setwd("D:/codigos/clustering")

vinos <- read.csv("vinos.csv")
head(vinos)
dim(vinos)
x <- vinos[, -1]
z <- scale(x)
```

- `x` contiene solo las 13 mediciones.
- `scale` resta la media y divide por la desviación muestral.
- El CSV ya está incluido en `D:/codigos/clustering`.

Datos y escala en Python

```
os.chdir(r"D:\codigos\clustering")
vinos = pd.read_csv("vinos.csv")
print(vinos.head())
print(vinos.shape)
x = vinos.drop(columns="codigo")
# ddof=1 usa la misma desviacion estandar muestral que scale() de R.
z = (x - x.mean()) / x.std(ddof=1)
```

- `drop` retira el identificador.
- `ddof=1` permite usar la misma escala muestral que en R.
- Conservamos `x` y `z` sin agregarles etiquetas de clúster.

Una etiqueta obtenida por el algoritmo no debe volver a entrar como predictor.

Distancia euclidiana: una cuenta

Consideremos dos observaciones ya estandarizadas:

$$x = (1, 2), \qquad c = (3, 1).$$

La distancia al cuadrado es

$$\|x - c\|^2 = (1 - 3)^2 + (2 - 1)^2 = 4 + 1 = 5.$$

La distancia euclidiana es $\sqrt{5} \approx 2.236$.

K-means minimiza la suma de distancias euclidianas al cuadrado.

El cuadrado aumenta el peso de observaciones muy alejadas del centro.

K-means: criterio de ajuste

Buscamos grupos $C_1, \ldots, C_k$ y centros $\mu_1, \ldots, \mu_k$ que reduzcan

$$WSS = \sum_{g=1}^{k} \sum_{i \in C_g} \|z_i - \mu_g\|^2.$$

Para un grupo dado, el centro que minimiza ese costo es su media:

$$\mu_g = \frac{1}{n_g} \sum_{i \in C_g} z_i.$$

Un WSS menor significa mayor compacidad bajo esta escala y este criterio.

No demuestra por sí solo que encontramos grupos reales o útiles.

Iteraciones de K-means

1. Fijar k y elegir los centros iniciales.
2. Asignar cada observación al centro más cercano.
3. Recalcular cada centro como la media de su grupo.
4. Repetir asignación y actualización hasta estabilizar el ajuste.

Cada paso reduce o mantiene el costo, pero la solución puede ser local.

Repetimos el procedimiento desde distintos puntos de partida.

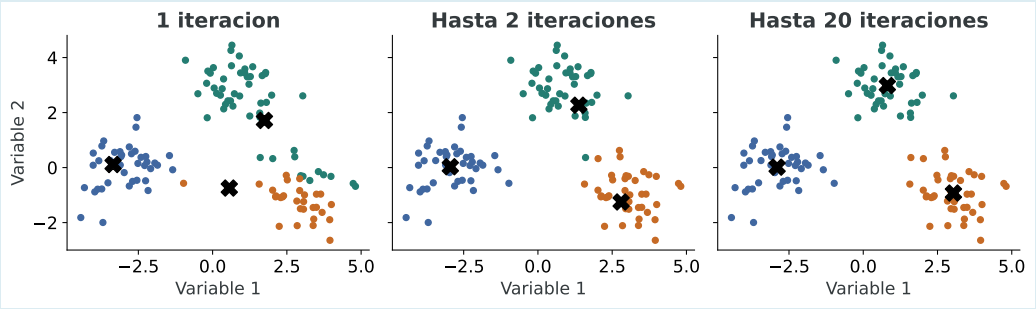
Una iteración resuelta

Datos: $1, 2, 3, 8, 9, 10$. Fijamos $k = 2$ y centros iniciales $c_1 = 1$, $c_2 = 8$.

x	$(x-1)^2$	$(x-8)^2$	Grupo
1	0	49	1
2	1	36	1
3	4	25	1
8	49	0	2
9	64	1	2
10	81	4	2

Nuevos centros: $\mu_1 = (1+2+3)/3 = 2$ y $\mu_2 = (8+9+10)/3 = 9$.

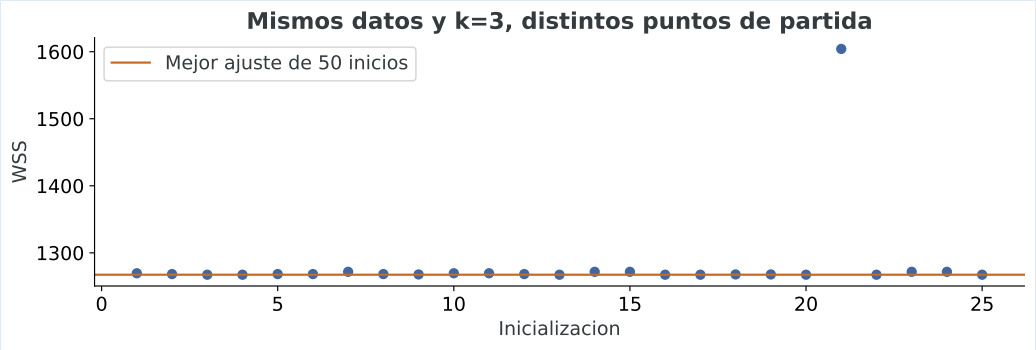
El ajuste en dos dimensiones



Las cruces cambian de posición al actualizar los centros.

Ejemplo simulado con los mismos datos e inicio. Los colores representan la asignación de cada ajuste.

La inicialización importa



Conservamos el ajuste de menor WSS entre varias inicializaciones.

Vinos estandarizados, $k = 3$. `nstart=50` en R y `n_init=50` en Python. Una semilla fija permite repetir el cálculo.

WSS, BSS y variabilidad total

$$TSS = \sum_i \|z_i - \bar{z}\|^2,$$
$$WSS = \sum_g \sum_{i \in C_g} \|z_i - \mu_g\|^2,$$
$$BSS = \sum_g n_g \|\mu_g - \bar{z}\|^2.$$

Para estas medias y las mismas observaciones: $TSS = WSS + BSS$.

BSS compara cada centro con la media global y pondera por el tamaño del grupo.

BSS no es el promedio de distancias entre todos los pares de centros.

WSS y BSS: cuenta completa

Datos: $1, 2, 3, 8, 9, 10$. Centros: 2 y 9. Media global: 5.5.

$$WSS = \underbrace{1 + 0 + 1}_{\{1,2,3\}} + \underbrace{1 + 0 + 1}_{\{8,9,10\}} = 4.$$

$$BSS = 3(2 - 5.5)^2 + 3(9 - 5.5)^2 = 36.75 + 36.75 = 73.5.$$

$$TSS = 20.25 + 12.25 + 6.25 + 6.25 + 12.25 + 20.25 = 77.5.$$

Verificación: $4 + 73.5 = 77.5$.

Al fijar los datos y la escala, reducir WSS equivale a aumentar BSS.

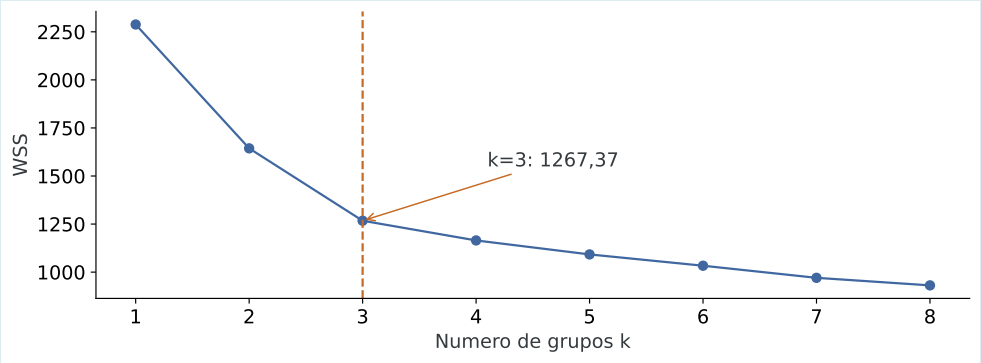
Elección del número de grupos

- **Codo**: buscar una disminución cada vez menor del WSS.
- **Silueta**: comparar cercanía al propio grupo y al grupo vecino.
- **Gap**: contrastar la compacidad con una referencia simulada.
- Revisar tamaños, estabilidad e interpretación de los grupos.

El menor WSS favorece más grupos. En el extremo, un grupo por fila da WSS cero.

El costo de ajuste necesita un criterio adicional para elegir k.

Codo en los vinos



$k = 3$ es un candidato interpretable. La curva sigue disminuyendo después.

Cálculo con el script Python, 13 variables estandarizadas. El codo no es una prueba formal ni garantiza un único candidato.

Silueta: interpretación y ejemplo

$a(i)$: distancia media a los otros puntos de su grupo.

$b(i)$: la menor distancia media a los puntos de otro grupo.

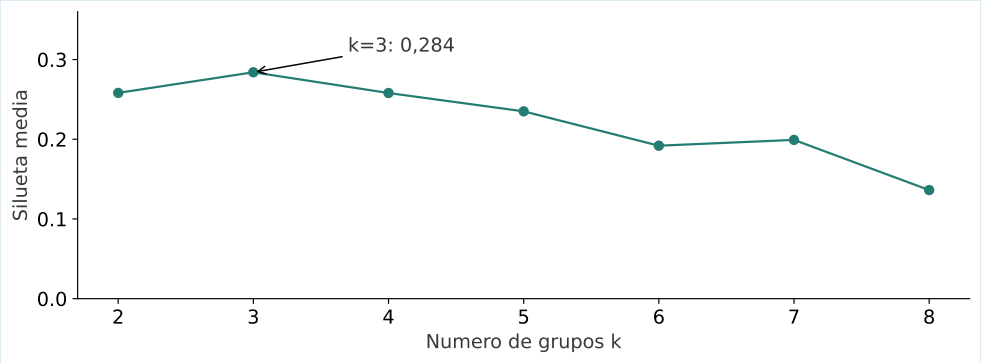
$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

Si $a(i) = 2$ y $b(i) = 6$, entonces $s(i) = (6 - 2)/6 = 0.667$.

- Cerca de 1: mayor separación respecto del grupo vecino.
- Cerca de 0: observación en una zona de superposición.
- Negativa: en promedio, está más cerca de otro grupo.

Se usa distancia euclidiana, no su cuadrado. Por convención, una observación sola en su grupo tiene silueta cero.

Silueta media en los vinos



El mayor valor entre $k = 2, \ldots, 8$ ocurre en $k = 3$: aproximadamente 0.284.

Es el mejor de estos candidatos, aunque persiste superposición entre grupos.

Cálculo Python sobre las 13 variables. No hay silueta para $k = 1$ porque falta un grupo vecino.

Gap: una referencia sin estructura de grupos

Comparamos el WSS observado con el de datos simulados en una región de referencia.

$$\mathrm{Gap}(k) = \frac{1}{B}\sum_{b=1}^{B} \log W_{kb}^{*} - \log W_k.$$

1. Ajustar K-means a los datos reales para cada k.
2. Simular B bases y calcular su WSS para los mismos valores de k.
3. Comparar el costo observado con el promedio en escala logarítmica.

Aquí usamos referencia uniforme en el rango de cada columna estandarizada.

Tibshirani, Walther y Hastie (2001). La referencia y la escala forman parte del criterio.

Gap: regla de un error estándar

Elegimos el primer k que cumple

$$\text{Gap}(k) \geq \text{Gap}(k+1) - s_{k+1}.$$

Ejemplo hipotético:

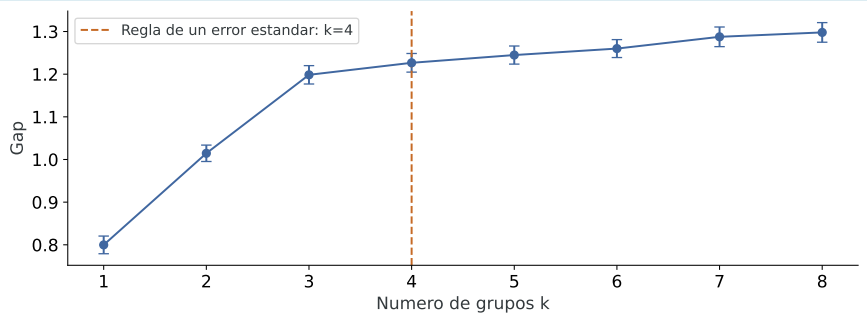
k	Gap	s_k
1	0.40	0.04
2	0.62	0.05
3	0.64	0.04

Para $k = 1$: $0.40 < 0.62 - 0.05 = 0.57$. No cumple.

Para $k = 2$: $0.62 \geq 0.64 - 0.04 = 0.60$. Elegimos 2.

Esta regla puede elegir un k menor que el máximo del Gap.

Cuando los criterios discrepan



Silueta: $k = 3$. Gap con esta referencia y 30 simulaciones: $k = 4$ en R y Python.

Trabajaremos con $k = 3$ para describir perfiles y reconoceremos $k = 4$ como alternativa.

Gráfico Python. Una simulación más extensa o una referencia distinta puede modificar la recomendación.

K-means y silueta en R

```r
set.seed(2026)
modelos <- vector("list", 8)
wss <- numeric(8)
silueta <- rep(NA_real_, 8)
distancias <- dist(z)

for (k in 1:8) {
  modelos[[k]] <- kmeans(z, centers = k, nstart = 50,
                         iter.max = 100, algorithm = "Lloyd")
  wss[k] <- modelos[[k]]$tot.withinss
  if (k > 1) {
    silueta[k] <- mean(silhouette(modelos[[k]]$cluster, distancias)[, 3])
  }
}
```

`modelos[[k]]` guarda el ajuste. La condición evita calcular silueta con un solo grupo.

K-means y silueta en Python

```
modelos = []
wss = []
silueta = []
for k in range(1, 9):
    modelo = KMeans(n_clusters=k, n_init=50, max_iter=100,
                    random_state=2026, algorithm="lloyd", tol=1e-8)
    modelo.fit(z)
    modelos.append(modelo)
    wss.append(modelo.inertia_)
    if k > 1:
        silueta.append(silhouette_score(z, modelo.labels_))
    else:
        silueta.append(np.nan)
```

`inertia_` es WSS. `labels_` contiene los grupos, numerados desde cero.

Los números de grupo son etiquetas

Con $k = 3$, ambos scripts encuentran la misma partición, con distinta numeración.

Grupo R	Grupo Python	Tamaño	Rasgo descriptivo
1	3	65	Menor alcohol y color
2	2	50	Mayor color y menor matiz
3	1	62	Más flavonoides y prolina

$WSS = 1267.37$, $BSS = 1020.63$, $TSS = 2288$.

El grupo 1 de R no tiene por qué ser el grupo 1 de Python.

En otros valores de k, también pueden cambiar las particiones.

Perfiles en la escala original

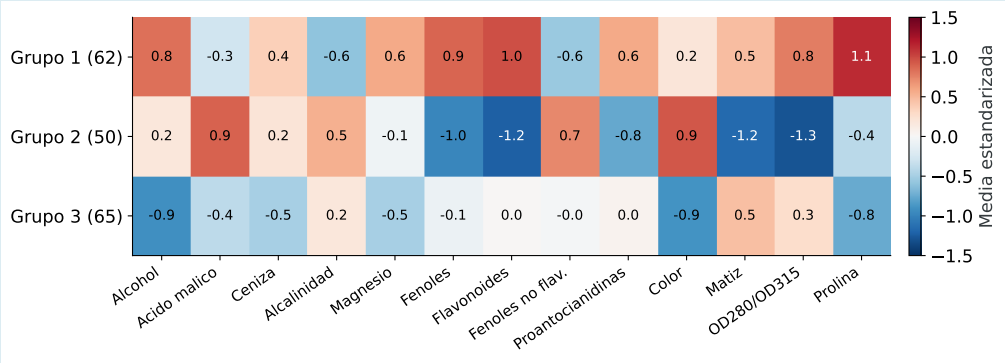
Grupo Python	*n*	Alcohol	Flavonoides	Color
1	62	13.68	3.00	5.45
2	50	13.14	0.82	7.18
3	65	12.25	2.05	2.97

Son promedios del grupo. No describen exactamente a todos sus integrantes.

- Comparar medias junto con distribuciones y tamaños.
- Volver a las unidades originales facilita la descripción.

Estas mediciones no permiten afirmar precio, calidad, preferencia ni potencial de guarda.

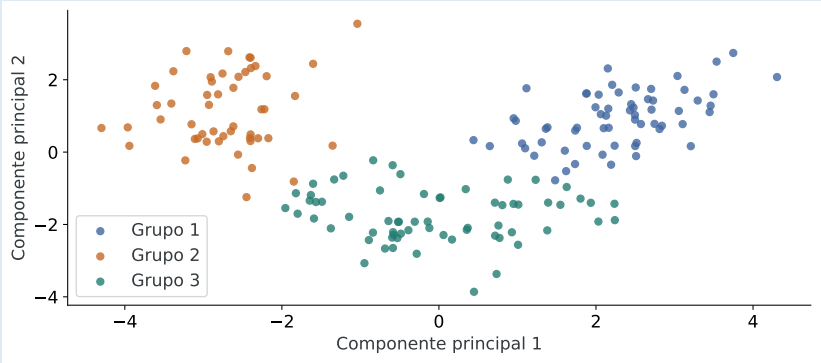
Perfiles estandarizados



Un valor positivo indica una media de grupo superior a la media general.

Etiquetas Python. Escala común en desviaciones estándar. Los colores resumen las mediciones, no una evaluación de calidad.

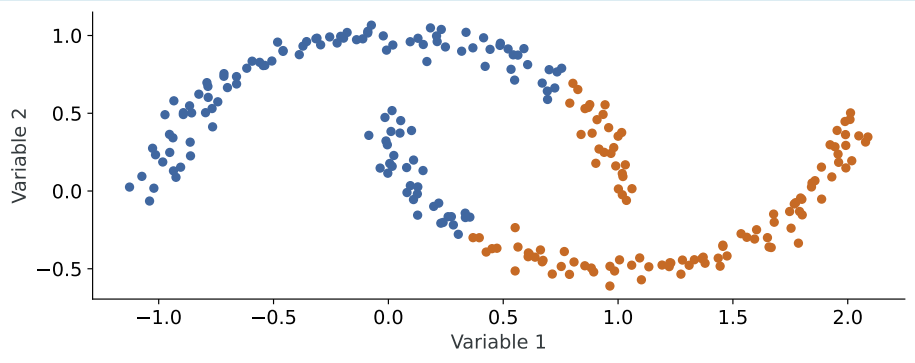
Una proyección de los grupos



PCA permite dibujar una aproximación en dos dimensiones.

El agrupamiento se ajustó con las 13 variables, no con estas dos coordenadas.

Limitaciones de K-means



K-means puede dividir estructuras curvas en grupos compactos que no siguen su forma.

Datos simulados. También conviene revisar atípicos, escalas, variables redundantes y grupos de tamaños o dispersiones muy distintos.

K-modes para variables categóricas

No tiene sentido promediar categorías como “App”, “Web” o “Sucursal”.

- El centro contiene la categoría más frecuente en cada variable.
- La disimilitud cuenta cuántas variables tienen valores diferentes.
- Se alternan asignación de observaciones y actualización de modas.

$$d(x, m) = \sum_{j=1}^{p} \mathbf{1}(x_j \neq m_j).$$

Cambiar los códigos 1, 2 y 3 por nombres no cambia esta disimilitud.

No se interpretan las diferencias numéricas entre esos códigos.

Desacuerdos: una asignación resuelta

Cliente: **(Gold, App, Sur)**.

Centro	Plan	Canal	Región	Desacuerdos
A	Gold	App	Norte	$0 + 0 + 1 = 1$
B	Silver	Web	Sur	$1 + 1 + 0 = 2$

El cliente se asigna al grupo A porque $1 < 2$.

Cada variable aporta un desacuerdo como máximo en esta versión no ponderada.

Las categorías no necesitan una codificación ordinal.

Actualización de una moda

Supongamos que un grupo contiene estos tres clientes:

Plan	Canal	Región
Gold	App	Norte
Gold	Web	Norte
Silver	App	Sur

La moda por columna es **(Gold, App, Norte)**.

$$\text{Costo del grupo} = 0 + 1 + 2 = 3 \text{ desacuerdos.}$$

Una moda es la categoría más frecuente. Puede haber empates y no siempre supera el 50 %.

Categorías para el ejemplo de vinos

Variable	Bajo	Medio	Alto
Alcohol	$x < 13$	No se usa	$x \geq 13$
Flavonoides	$x < 1.5$	$1.5 \leq x < 3$	$x \geq 3$
Color	$x < 4$	$4 \leq x < 6.5$	$x \geq 6.5$

Conservamos los cortes del material para ilustrar K-modes.

- Son cortes docentes, no umbrales de calidad enológica.
- Categorizar pierde información de las mediciones originales.
- Aquí las categorías se comparan por igualdad, sin usar su orden.

Categorías en R

```
categorias <- data.frame(
  alcohol = factor(ifelse(vinos$alcohol < 13, "bajo", "alto")),
  flavonoides = cut(vinos$flavonoides, c(-Inf, 1.5, 3, Inf),
                    labels = c("bajo", "medio", "alto"), right = FALSE),
  color = cut(vinos$color, c(-Inf, 4, 6.5, Inf),
              labels = c("bajo", "medio", "alto"), right = FALSE)
)
print(head(categorias))
```

`right=FALSE` forma intervalos cerrados por la izquierda.

Por ejemplo, un valor de flavonoides igual a 1.5 pertenece a “medio”.

Categorías en Python

```
categorias = pd.DataFrame({
    "alcohol": np.where(vinos.alcohol < 13, "bajo", "alto"),
    "flavonoides": np.where(vinos.flavonoides < 1.5, "bajo",
                            np.where(vinos.flavonoides < 3, "medio", "alto")),
    "color": np.where(vinos.color < 4, "bajo",
                      np.where(vinos.color < 6.5, "medio", "alto"))
})
print(categorias.head())
```

Las condiciones reproducen exactamente los mismos cortes que en R.

K-modes admite estas etiquetas directamente. No necesitamos LabelEncoder.

K-modes en R: varias inicializaciones

```
ajustar_modos <- function(k) {
  mejor <- kmodes(categorias, modes = k, iter.max = 100, weighted = FALSE)
  for (inicio in 2:20) {
    candidato <- kmodes(categorias, modes = k, iter.max = 100, weighted = FALSE)
    if (sum(candidato$withindiff) < sum(mejor$withindiff)) {
      mejor <- candidato
    }
  }
  mejor
}
```

`withindiff` contiene un costo por grupo. `sum` obtiene el total.

La función conserva el mejor de 20 ajustes, con el mismo k.

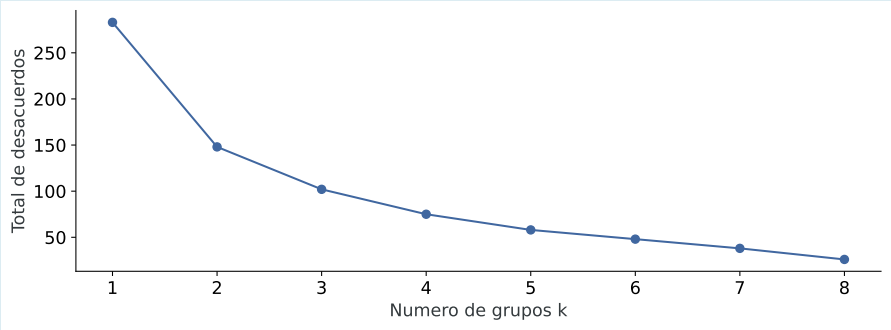
K-modes en Python

```
modelos_modos = []
costo_modos = []
for k in range(1, 9):
    modelo = KModes(n_clusters=k, init="Huang", n_init=20,
                    max_iter=100, random_state=2026)
    modelo.fit(categorias)
    modelos_modos.append(modelo)
    costo_modos.append(modelo.cost_)

kmodas = modelos_modos[3]
grupo_modos = kmodas.labels_ + 1
```

`n_init=20` realiza las inicializaciones. `cost_` es el costo total.

Costo de K-modes



El costo cuenta desacuerdos. No es una suma de cuadrados.

Usaremos $k = 4$ para ilustrar perfiles, sin afirmar que sea el único valor adecuado.

Resultados Python, cortes definidos antes del ajuste y 20 inicializaciones por k.

Lectura de un perfil categórico

Ejemplo Python, grupo 2 de K-modes: $n = 48$.

Color	Número	Proporción
Alto	40	$40/48 = 83.3\,\%$
Bajo	1	$1/48 = 2.1\,\%$
Medio	7	$7/48 = 14.6\,\%$

Su centro modal es (alcohol alto, flavonoides bajos, color alto).

El perfil resume frecuencias. No significa que todos los vinos tengan el mismo color.

R puede repartir los empates de otra forma, incluso con el mismo costo total.

K-prototypes para variables mixtas

Combinamos una parte numérica con otra categórica:

$$d(x, q) = \sum_{j \in N} (x_j - \mu_j)^2 + \gamma \sum_{j \in C} \mathbf{1}(x_j \neq m_j).$$

- N: variables numéricas estandarizadas.
- C: variables categóricas.
- q: prototipo con medias y modas.
- $\gamma > 0$: peso de los desacuerdos categóricos.

El equilibrio depende del peso y también del número de variables de cada tipo.

Costo mixto: una cuenta resuelta

Una observación tiene dos valores numéricos $(1, 2)$ y categorías (App, Sur).

Prototipo	Parte numérica	Categorías	Costo
A	$(0, 2)$	(App, Norte)	$1 + \gamma$
B	$(2, 3)$	(App, Sur)	2

Para A: $(1-0)^2 + (2-2)^2 = 1$ y un desacuerdo.

Para B: $(1-2)^2 + (2-3)^2 = 2$ y cero desacuerdos.

$$\gamma = 0.5: \quad 1.5 < 2 \Rightarrow A; \qquad \gamma = 3: \quad 4 > 2 \Rightarrow B.$$

Modificar el peso puede cambiar la asignación, aunque los datos sean los mismos.

Datos mixtos en R

```
mixtos <- data.frame(z[, c("ceniza", "matiz")], categorias)
peso <- 1
```

Numéricas	Ceniza y matiz, estandarizados
Categóricas	Alcohol, flavonoides y color

Las columnas categóricas deben ser factores en `clustMixType`.

No incluimos cada medición a la vez como número y como categoría.

Así evitamos dar peso duplicado a la misma información.

Datos mixtos en Python

```
mixtos = pd.concat([z[["ceniza", "matiz"]], categorias], axis=1)
peso = 1
```

Índice	Variable	Tipo
0	Ceniza	Numérica
1	Matiz	Numérica
2	Alcohol	Categórica
3	Flavonoides	Categórica
4	Color	Categórica

El ajuste usa `categorical=[2, 3, 4]` porque Python cuenta desde cero.

K-prototypes en R

```
set.seed(2026)
modelos_proto <- vector("list", 8)
costo_proto <- numeric(8)
for (k in 1:8) {
  modelos_proto[[k]] <- kproto(mixtos, k = k, lambda = peso,
                               nstart = 20, iter.max = 100, verbose = FALSE)
  costo_proto[k] <- modelos_proto[[k]]$tot.withinss
}

kprototipos <- modelos_proto[[3]]
print(table(kprototipos$cluster))
print(kprototipos$centers)
```

Aquí `lambda` es el peso de las categorías. No es la penalización de Ridge.

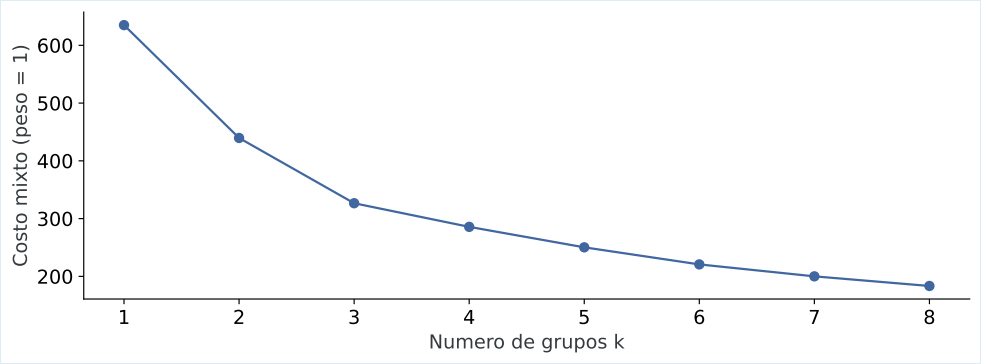
K-prototypes en Python

```
modelos_proto = []
costo_proto = []
for k in range(1, 9):
    modelo = KPrototypes(n_clusters=k, init="Huang", n_init=20,
                         gamma=peso, max_iter=100, random_state=2026)
    modelo.fit(mixtos.to_numpy(), categorical=[2, 3, 4])
    modelos_proto.append(modelo)
    costo_proto.append(modelo.cost_)

kprototipos = modelos_proto[2]
grupo_proto = kprototipos.labels_ + 1
```

El mismo peso se llama `gamma` en esta implementación.

Costo de K-prototypes



Comparamos candidatos con las mismas variables, escala y peso $\gamma = 1$.

Para la descripción usamos una solución de ejemplo con $k = 3$.

Resultados Python. Un costo menor al aumentar k no basta para justificar más grupos.

Un prototipo combina medias y modas

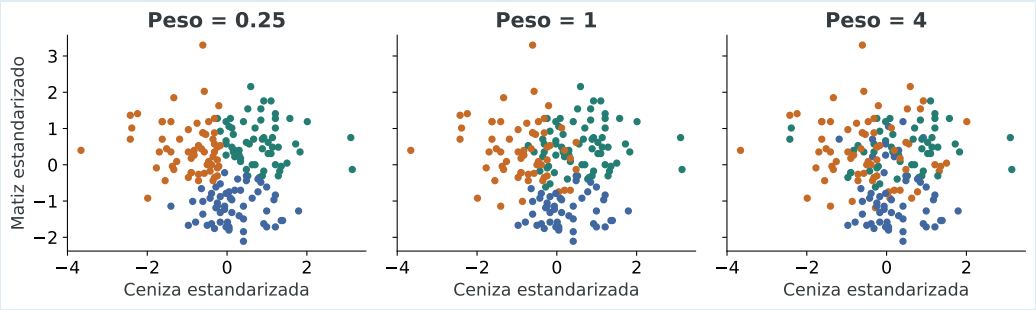
Resultado Python, $k = 3$ y peso 1:

Grupo	n	Ceniza z	Matiz z	Alcohol	Flavon.	Color
1	69	0.625	0.506	Alto	Medio	Medio
2	47	0.225	-1.258	Alto	Bajo	Alto
3	61	-0.881	0.396	Bajo	Medio	Bajo

En el grupo 2, matiz está 1.258 desviaciones bajo la media general. “Color alto” es la moda, pero solo $30/47 \approx 63.8\,\%$ tiene esa categoría.

Un prototipo puede no corresponder a ninguna observación real.

Sensibilidad al peso de las categorías



Cambian las particiones al cambiar el peso, manteniendo $k = 3$.

Ilustración Python con dos variables numéricas y tres categóricas. Solo se muestran ceniza y matiz. Colores alineados por el centro de matiz para facilitar la lectura.

Evaluación sin una respuesta observada

- No calculamos exactitud ni AUC: no hay una clase observada que predecir.
- Usamos criterios internos acordes con las variables y la disimilitud.
- Revisamos si los perfiles reaparecen al cambiar inicios o remuestrear.
- Contrastamos su utilidad con el problema que motivó el agrupamiento.

Esta exploración usa las 177 filas. No requiere automáticamente una división 70/30.

Para asignar casos futuros, se fijan el escalamiento y los centros con los datos de desarrollo.

La evaluación de ese uso futuro necesita un diseño separado.

Comparación entre los tres métodos

Método	k	Costo del ejemplo	Significado
K-means	3	1267.37	Distancias al cuadrado
K-modes	4	75	Desacuerdos
K-prototypes	3	326.64	Costo mixto, peso 1

No elegimos K-modes porque 75 sea el menor número de esta tabla. Los datos de entrada, las unidades y las funciones de costo son distintos.

La comparación pertinente comienza por el tipo de variables y el propósito.

Interpretación de una segmentación

Antes de usar los grupos, revisamos:

1. Variables incluidas, escalas y posibles observaciones atípicas.
2. Tamaños, perfiles y variación dentro de cada grupo.
3. Sensibilidad a k, inicialización y peso de las categorías.
4. Si las conclusiones describen las mediciones o exceden la evidencia.

Un agrupamiento organiza la información. Su utilidad debe justificarse en el contexto.

Material y referencias

- Material base: *Clase 34*, Eduardo Rodríguez Faúndez; y *Clase 4: Algoritmos de segmentación*, Acosta, Castro y Moraga.
- Wine, UCI. `https://doi.org/10.24432/C5PC7J`. Licencia CC BY 4.0.
- R: `stats::kmeans`, `cluster::silhouette` y `cluster::clusGap`.
- `klaR::kmodes`. Szepannek (2018), `clustMixType`.
- Python: scikit-learn y `kmodes`, Nelis J. de Vos.
- Tibshirani, Walther y Hastie (2001). *Estimating the number of clusters in a data set via the gap statistic*.

Complemento: Gap en R

```
set.seed(2026)
gap <- clusGap(z, FUNcluster = kmeans, K.max = 8, B = 30,
               nstart = 20, iter.max = 100, algorithm = "Lloyd",
               d.power = 2, spaceH0 = "original", verbose = FALSE)
print(round(gap$Tab[, c("gap", "SE.sim")], 3))
k_gap <- maxSE(gap$Tab[, "gap"], gap$Tab[, "SE.sim"], method = "Tibs2001SEmax")
print(k_gap)
```

- `B=30`: número de bases de referencia simuladas.
- `d.power=2`: criterio cuadrático.
- `spaceH0="original"`: referencia en el rango de cada columna de `z`.

Complemento: Gap en Python

```
rng = np.random.default_rng(2026)
B = 30
log_wss_ref = np.zeros((B, 8))
for b in range(B):
    referencia = rng.uniform(z.min().to_numpy(), z.max().to_numpy(), size=z.shape)
    for k in range(1, 9):
        modelo = KMeans(n_clusters=k, n_init=20, max_iter=100,
                        random_state=2026 + b, algorithm="lloyd", tol=1e-8)
        modelo.fit(referencia)
        log_wss_ref[b, k - 1] = np.log(modelo.inertia_)
```

`log_wss_ref` guarda un costo para cada simulación y cada k.

Complemento: la regla en Python

```
gap = log_wss_ref.mean(axis=0) - np.log(wss)
error_gap = log_wss_ref.std(axis=0, ddof=1) * np.sqrt(1 + 1 / B)
k_gap = 8
for k in range(1, 8):
    if gap[k - 1] >= gap[k] - error_gap[k]:
        k_gap = k
        break
print(pd.DataFrame({"k": range(1, 9), "Gap": gap, "Error": error_gap}).round(3))
print("k segun la regla de un error estandar:", k_gap)
```

Si ninguna comparación anterior cumple, conservamos el máximo k explorado.

R y Python usan la misma definición y referencia, pero distintas secuencias aleatorias. Se esperan pequeñas diferencias numéricas.